

MINI PROJECT

-Software Engineering

Title : Hostel Management System

Batch No. -

G Bharath	1CD23IS046
Hemanth K S	1CD23IS058
Basavaraj	1CD23IS020
Gahan Krishna K	1CD23IS049
Chirag H S	1CD23IS036
Amith K N	1CD23IS007



PROBLEM STATEMENT

Managing a hostel manually involves complex record-keeping of room availability, student details, and visitor logs.

Traditional systems are prone to:

- Data Redundancy: Multiple entries for the same student or room across different register
- Inaccuracy: Errors in calculating vacancy or tracking visitor exit times.

- **Slow Processing:** Manual room allocation is time-consuming and lacks transparency for students.
- **Lack of Security:** Physical registers are easily accessible and difficult to back up.

ABSTRACT

The Hostel Room Allocation System is a web-based management solution designed to automate the administrative tasks of a hostel. Developed using the Node.js

ecosystem and SQLite, it provides a centralized platform where administrators can manage hostel infrastructure, approve student applications, and monitor visitor activity. Students can register online and track their allocation status, ensuring a transparent and efficient workflow.

OBJECTIVES

- Automation: Replace manual room allocation with a logic-driven automated approval process
- Data Integrity: Use a relational database to ensure consistent and accurate records for hostels, rooms, and students.

- Transparency: Provide students with a dedicated dashboard to view their registration status and allocated room details.
- Resource Tracking: Maintain real-time logs of visitors and their
- association with residents.

STAKEHOLDERS

- Administrator (Hostel Warden/Manager): Responsible for managing room inventory, approving/rejecting student registrations, and monitoring visitor
- Students: Users who apply for accommodation, login to view status, and manage their personal profile.
- Security Personnel: (Secondary) Users who record visitor entries and exits via the system.

— TEAM MEMBERS & ROLES

G Bharath
Gahan Krishna K
Hemanth K S
Amith K N
Chirag H S
Basavaraj

Team Lead
Scrum Master
Developer
Product Owner
Designer
Tester

SYSTEM OVERVIEW / PROPOSED SOLUTION

The proposed system is a multi-user web application that digitizes the entire hostel lifecycle:

1. Registration: Students apply with their academic and contact details.
2. Approval Workflow: Admins review "Pending" applications and approve them based on real-time room capacity.

3. Allocation: The system automatically identifies vacant rooms and assigns them upon approval.
4. Visitor Management: A dedicated module tracks outsiders visiting students, including timestamped logs.

SYSTEM ARCHITECTURE

The system follows a Model-View-Controller (MVC) influenced architecture:

- * Client (View): Interactive UI built with EJS (Embedded JavaScript) templates and styled with CSS.
- * Server (Controller): Node.js with the Express.js framework handles routing, business logic, and session management.

* Database (Model): SQLite3 serves as the persistent storage layer, managing relational data via SQL queries.

TECHNOLOGY STACK

Frontend:

- HTML5, CSS3, JavaScript

Backend:

- Node.js

- Express.js

Database:

- SQLite (Relational Database Management System)

Session Management:

- *express-session for secure user authentication.

Environment:

- *NPM (Node Package Manager) for dependency

TECHNOLOGY STACK

Functional Requirements:

- Secure Login for Admins and Students.

- CRUD (Create, Read, Update, Delete) operations for Hostels and Rooms.
- Student registration and status tracking (Pending/Approved/Rejected).
- Automatic calculation of room occupancy.

Non-Functional Requirements:

- Security: Session-based authentication to prevent unauthorized access to admin routes.
- Reliability: Database constraints (Foreign Keys) to ensure data consistency.

- Usability: Minimalist UI for easy navigation by non-technical users

USE CASE

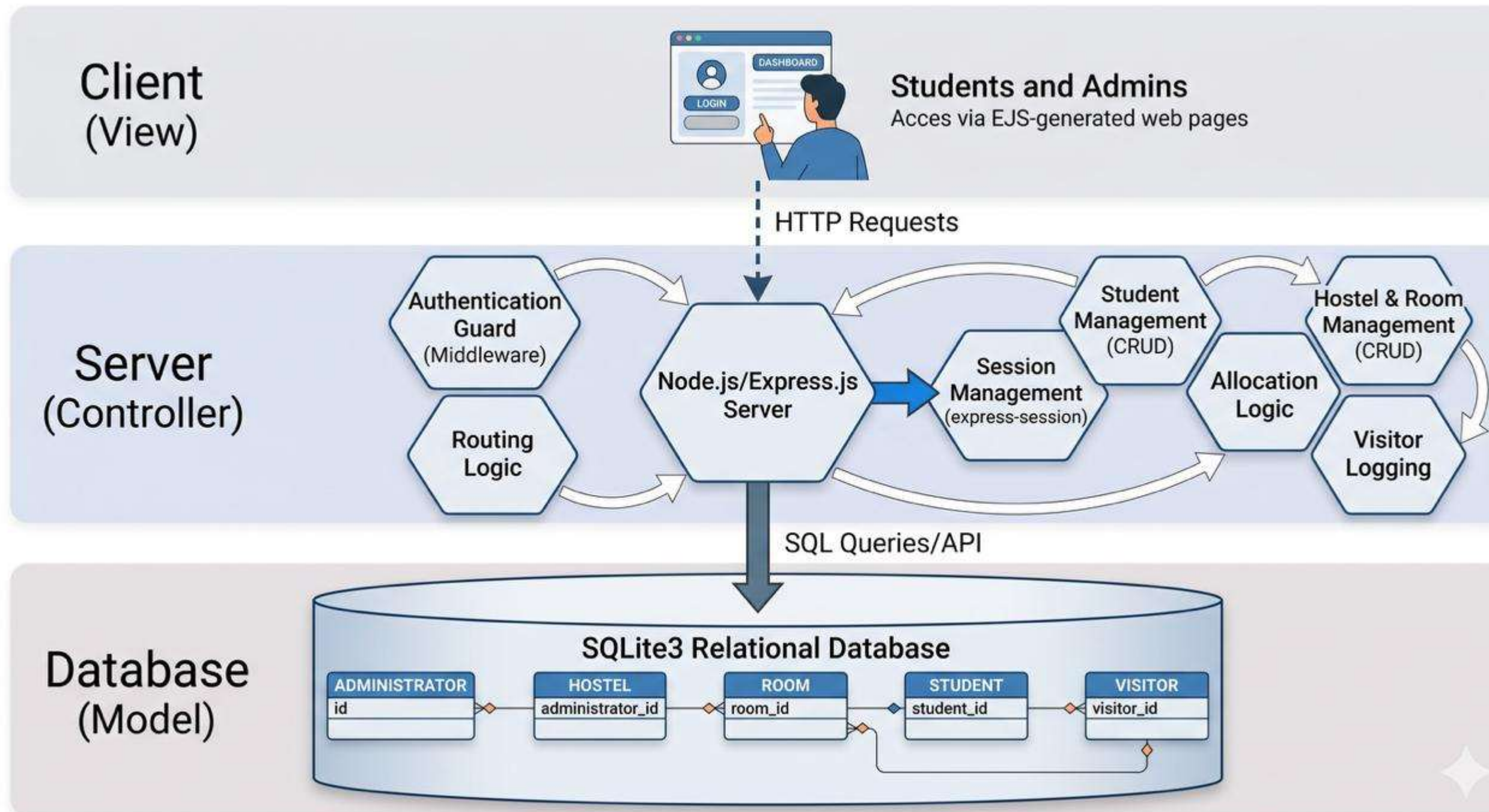
Actors:

Admin: Add/Delete Hostels, Modify Room Capacity, Approve, Applications, View Student List, Log Visitors.

Student : Self-Registration, Login, View Dashboard, Check Room
Number.

SYSTEM DESIGN

System Logic: When an Admin approves a student, the system queries the ROOM table for rooms where $\text{current_occupancy} < \text{capacity}$ and updates the STUDENT status.



AGILE METHODOLOGY

Method Used: Agile (Scrum)

- Development in small sprints
- Daily team coordination
- Continuous testing and improvement
- Regular reviews after each sprint

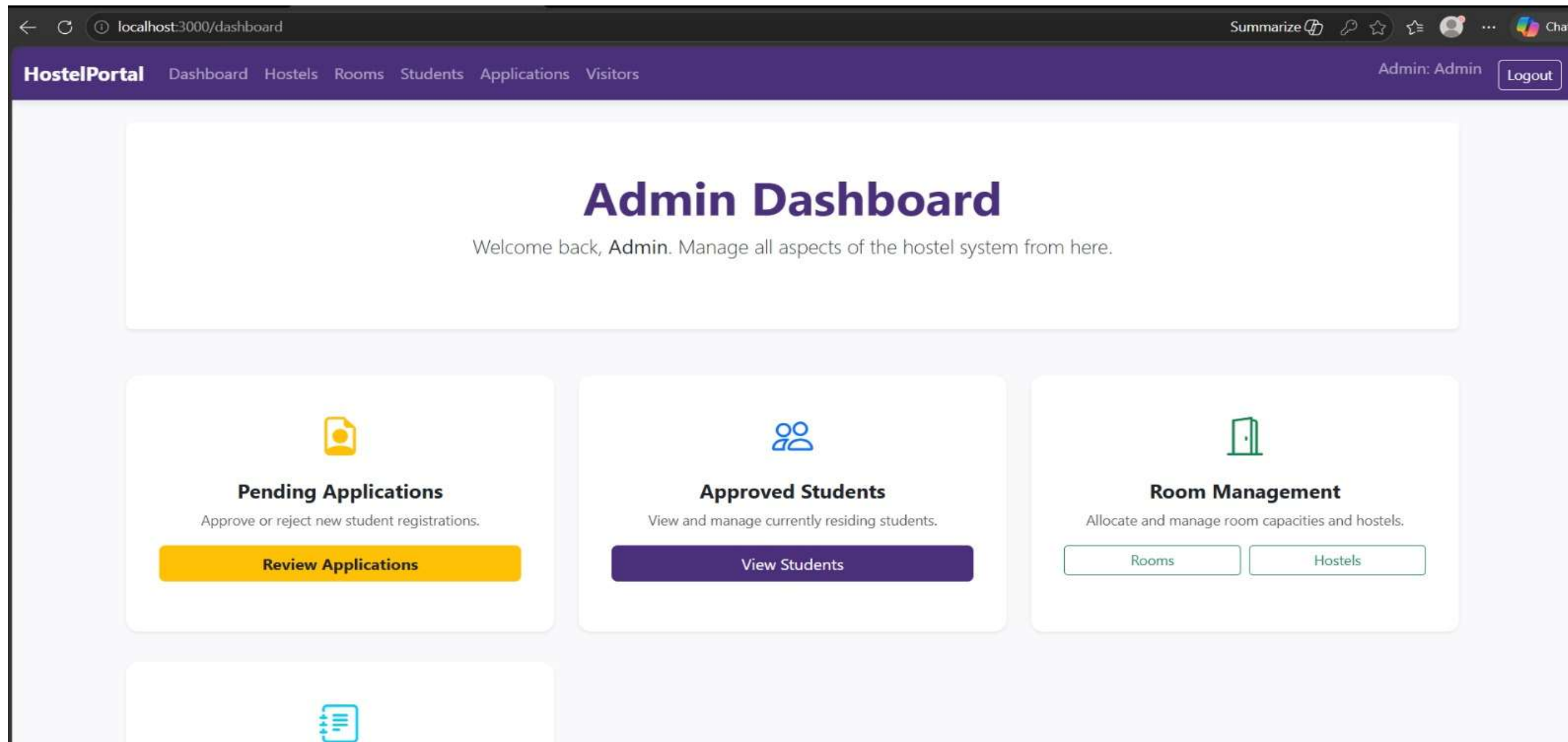
Tools Used:

- Jira for task tracking

IMPLEMENTATION

- * Routing: Modular routes (e.g., routes/students.js) to keep logic clean.
- * Database Schema: Tables for ADMINISTRATOR, HOSTEL, ROOM, STUDENT, and VISITOR with established relations.

* Middleware: Custom authentication guards (isAuthenticated) to protect sensitive routes.



TESTING APPROACH

Types of Testing:

- Unit Testing : Testing individual SQL queries (e.g., checking if `getAvailableRooms` returns the correct count).
- Integration Testing : Verifying that a student registration form correctly inserts data into the SQLite database.
- System Testing : Validating the full "Application to Allocation" flow,

- ensuring a student cannot be assigned to a full room.

TOOLS USED

- Development: Visual Studio Code (VS Code).
- Runtime: Node.js Runtime Environment.
- Database Management: SQLite3 CLI / SQLite Browser.

- Version Control: Git (if used) for tracking changes.
- Jira

CHALLENGES FACED

- Data Synchronization: Ensuring that room occupancy counts update immediately upon student approval.

- Session Persistence: Handling user session across different browser tabs while maintaining security
- Join Queries: Writing complex SQL JOINS to display student details along with their hostel and room names in a single view.

Conclusion

The Hostel Room Allocation System successfully transitions hostel management from a manual, error-prone process to an automated, secure digital platform.

By leveraging the speed of Node.js and the simplicity of SQLite, the system provides a robust tool for administrators while enhancing the experience for students. Future enhancements could include online fee payment integration and automated email notifications.

Future Enhancements:

- Authentication system

- Mobile application
- Cloud deployment
- AI-based blood demand prediction

Thank You